Experiment Name: False Position Method in Python

1. Objectives:

- To understand the concept of the False Position Method for finding roots of equations.
- To learn how this method gradually finds the root step by step within the set tolerance.
- To understand why it is a breaking method.

2. Introduction:

The False Position Method**,** or Regula Falsi**,** is a way to approximate the root of a nonlinear equation. We start with an interval [a,b][a, b][a,b] where the function changes sign and draw a straight line between (a,f(a))(a, f(a))(a,f(a)) and (b,f(b))(b, f(b))(b,f(b)); the x-intercept of this line gives a better guess for the root. You then update the interval based on the sign at this point and repeat until the function value is small enough. Unlike the Bisection Method, it often converges faster because it uses a weighted estimate instead of the simple midpoint. But here's the catch—sometimes one endpoint barely moves while the other creeps forward, so the root hardly improves. This slow, stubborn behavior is why people call it a "breaking" method: it can stall for many iterations, making the process feel stuck even though, in theory, it should work efficiently.

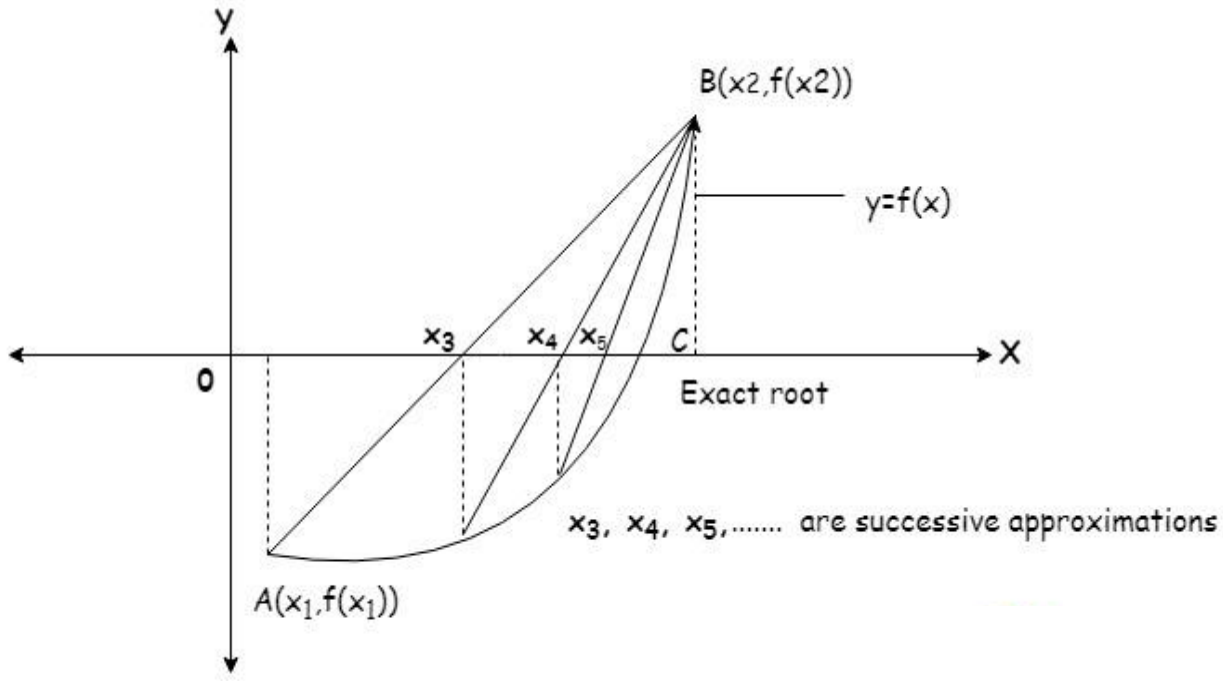

Fig 1: False Position Method

3. Procedure

Algorithm: False Position Method in Python

1. Start.
2. Define the function f(x).
3. Choose initial points a and b such that f(a) * f(b) < 0.
4. Calculate c using the False Position formula.
5. Evaluate f(c).
6. Check tolerance:
7. If |f(c)| ≤ tolerance, stop; c is the root.
8. Else, update the interval according to the sign of f(c).
9. Repeat steps 4–6 until the maximum number of iterations or the tolerance is satisfied.
10. Print the approximate root.
11. Stop.

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** PyCharm Community Edition 20251.1.1

5. Experiment Implementation:

Code:

```
def f(x):
    return x ** 3 - 2 * x - 5
a = 2
b = 3
tol = 0.00005
max = 100

def false_position(a, b, tol, max):
    if (f(a) * f(b) >= 0):
        print("Invalid interval")
        return None

print(f"{'Iteration':<12}{'a':<12}{'b':<12}{'f(a)':<12}{'f(b)':<
12}{'c':<12}{'f(c)':<12}")
    print("-" * 82)

    i = 1
    while i <= max:
        fa = f(a)
        fb = f(b)
        c = (a * fb - b * fa) / (fb - fa)
        fc = f(c)

print(f"{i:<12}{a:<12.6f}{b:<12.6f}{fa:<12.6f}{fb:<12.6f}{c:<12.
6f}{fc:<12.6f}")


        if abs(fc) <= tol:
            return (a * fb - b * fa) / (fb - fa)
        elif fa * fc < 0:
            b = c
        else:
            a = c
        i += 1
    return (a * fb - b * fa) / (fb - fa)

root = false_position(a, b, tol, max)
if root is not None:
    print(f"\nRoot is : {root:.6f}")
```

Output:

Iteration	a	b	f(a)	f(b)	c	f(c)
1	2.000000	3.000000	-1.000000	16.000000	2.058824	-0.390800
2	2.058824	3.000000	-0.390800	16.000000	2.081264	-0.147204
3	2.081264	3.000000	-0.147204	16.000000	2.089639	-0.054677
4	2.089639	3.000000	-0.054677	16.000000	2.092740	-0.020203
5	2.092740	3.000000	-0.020203	16.000000	2.093884	-0.007451
6	2.093884	3.000000	-0.007451	16.000000	2.094305	-0.002746
7	2.094305	3.000000	-0.002746	16.000000	2.094461	-0.001012
8	2.094461	3.000000	-0.001012	16.000000	2.094518	-0.000373
9	2.094518	3.000000	-0.000373	16.000000	2.094539	-0.000137
10	2.094539	3.000000	-0.000137	16.000000	2.094547	-0.000051
11	2.094547	3.000000	-0.000051	16.000000	2.094550	-0.000019

Root is : 2.094550

5. Discussion & Conclusion:

In this experiment, the False Position Method was successfully implemented in Python to approximate the root of a nonlinear equation. By using a straight line between the endpoints instead of the midpoint, the method steadily narrowed down the root, demonstrating a systematic approach to problems where exact solutions are difficult. It proved to be simple, reliable, and often efficient, converging within the given tolerance. While some iterations were needed, the method clearly showed how comparing function values and updating the interval step by step allows for accurate root approximation, highlighting the practical power of numerical methods in solving otherwise analytically challenging equations.